Publico, Marco R

Capstone Project

STEP 1: PROBLEM UNDERSTANDING AND FRAMING

Introduction

Suicide among military personnel has become an increasingly urgent mental health issue worldwide. Soldiers are often exposed to extreme stressors such as combat trauma, isolation, repeated deployment, and loss of comrades, which can lead to depression, post-traumatic stress disorder (PTSD), and suicide ideation. Despite existing prevention programs, many cases go undetected due to the stigma surrounding mental health help-seeking within military culture (Bryan et al., 2020).

In recent years, AI and ML technologies have demonstrated strong potential in mental health research. Algorithms can analyze large volumes of data, including speech, text, and physiological signals, to identify subtle emotional and cognitive patterns that may indicate distress (Guntuku et al., 2019). These tools can augment traditional psychological assessments by providing early warning signals to counselors, enabling more proactive intervention.

This study proposes an AI-based framework to detect suicide ideation using linguistic text among military personnel. The project integrates the counselor's perspective and ethical considerations to ensure responsible and human-centered use of AI.

Statement of the Problem

Traditional mental health monitoring in military settings relies on periodic assessments or self-report questionnaires. These methods often fail to capture real-time emotional changes and depend heavily on personal disclosure. Soldiers experiencing suicidal thoughts may withhold information due to fear of stigma or repercussions on their careers (Kuehn, 2020).

This brings about the solution of using AI and ML to identify early indicators of suicide ideation using text-based data from military personnel.

Objectives of the Study

To design and evaluate an AI-based system capable of detecting suicide ideation among military personnel using text-based data.

Specific Objectives

1. To identify linguistic markers associated with suicide ideation in military contexts
2. To develop and train an NLP-based ML model to classify text as indicating suicidal ideation or non-risk.
3. To validate the model's performance using established metrics (**precision, recall, F1 Score**)
4. To establish an ethical framework for deploying AI-assisted suicide risk detection in military mental health programs

Task Type

This project is framed as a binary text classification task, where the input is unstructured natural language (Reddit post text) and the output is a binary label indicating the presence (1) or absence (0) of suicidal ideation. This falls under supervised learning, sepcifically natural language processing (NLP) based classification, as opposed to regression, recommendation, anomaly detection, or clustering.

Publico, Marco R

Success Metrics and Institutional KPIs

Model performance is evaluated using accuracy, precision, recall, F1-score, and ROC-AUC. Given the asymmetric cost of prediction errors in this domain, recall is prioritized as the primary decision metric: a false negative (a missed case of genuine suicidal ideation) carries substantially greater real-world consequence than a false positive (a false alarm subject to routine human review).

Institutional Outcomes

As this project addresses a military mental health application rather than a commercial one, the equivalent of a "business KPI" is framed in institutional rather than financial terms. The applicable outcomes are: (1) reduction in undetected suicide risk cases among military personnel, (2) augmentation of counselor triage capacity by flagging high-risk text for review rather than replacing clinical judgment, and (3) demonstration of a defensible, ethically-audited AI framework suitable for further validation before any operational consideration within AFP mental health programs.

STEP 2: DATA COLLECTION AND UNDERSTANDING

Source and Purpose

This project uses the Suicidal Ideation Reddit Annotated, a publicly-sourced dataset of user posts, sourced from mental health-related subreddits, specifically annotated with binary labels indicating the presence or absence of suicidal ideation. The dataset was selected as a public, pre-existing, and already-anonymized source, avoiding the ethical concerns associated with primary data collection on sensitive mental health topics. The dataset was downloaded from Kaggle.com.

Size and Structure

The dataset contains 12,656 rows and 2 columns (usertext, label). After removing rows with missing text, 12,615 rows remained usable for analysis.

Data Dictionary

Variable	Type	Unites	Allowed Values	Notes
usertext	string	None	Any sting	Reddit post content. 41 rows have missing values. Length ranges from 1 to ~21,000 characters (0 to 3,868 words).
Label	Integer	None	0 or 1	1 = suicidal ideation (6,609 rows) 0 = non-risk (6,047 rows)

Publico, Marco R

STEP 3: DATA PREPROCESSING, APPLIED EDA & FEATURE ENGINEERING

a. Exploratory Data Analysis

- **Domain-derived Features**

```
import matplotlib.pyplot as plt
import seaborn as sns

# --- Domain-derived features ---
df['word_count'] = df['usertext'].apply(lambda x: len(str(x).split()))
df['char_count'] = df['usertext'].apply(lambda x: len(str(x)))
first_person_words = {'i', 'me', 'my', 'mine', 'myself'}
df['first_person_count'] = df['processed_text'].apply(
    lambda x: sum(1 for word in x.split() if word in first_person_words))

print("Domain-derived features added: word_count, char_count, first_person_count")
display(df[['word_count', 'char_count', 'first_person_count']].describe())
```

	word_count	char_count	first_person_count
count	12656.000000	12656.000000	12656.000000
mean	155.018173	813.894674	0.018726
std	226.159775	1192.933368	0.144588
min	0.000000	0.000000	0.000000
25%	29.000000	156.000000	0.000000
50%	84.000000	448.500000	0.000000
75%	191.000000	998.000000	0.000000
max	3868.000000	20972.000000	2.000000

```
# Label distribution
sns.countplot(data=df, x='label', ax=axes[1])
axes[1].set_title('Class Distribution (0=Non-Risk, 1=Suicidal Ideation)')


plt.tight_layout()
plt.savefig('/content/eda_distributions.png', dpi=150)
plt.show()

# Relationship check: average word count and first-person usage by label
print("\nAverage word count by label:")
display(df.groupby('label')['word_count'].mean())

print("\nAverage first-person pronoun count by label:")
display(df.groupby('label')['first_person_count'].mean())

# Word count distribution by label
```

Publico, Marco R

```
sns.histplot(data=df, x='word_count', hue='label', bins=50, ax=axes[0], element='step')
axes[0].set_xlim(0, 500)
axes[0].set_title('Word Count Distribution by Label')
```

Average word count by label:

	word_count
label	
0	82.354556
1	221.502799

```
# First-person pronoun usage by label
sns.boxplot(data=df, x='label', y='first_person_count', ax=axes[2])
axes[2].set_ylim(0, 30)
axes[2].set_title('First-Person Pronoun Usage by Label')
```

Average first-person pronoun count by label:

	first_person_count
label	
0	0.011080
1	0.025722

```
# Check class distribution of the 'label' column
print('\nClass distribution of the label column:')
display(df[‘label'].value_counts())
```

Class distribution of the label column:

	count
label	
1	6609
0	6047

```
# Display the percentage distribution as well
print('\nClass distribution (percentages):')
display(df['label'].value_counts(normalize=True) * 100)
```

Publico, Marco R

Class distribution (Percentages)

	proportion
label	
1	52.220291
0	47.779709

- **Class/Label Distribution**

```
# Check class distribution of the 'label' column
print('\nClass distribution of the label column:')
display(df['label'].value_counts())
```

	count
label	
1	6609
0	6047

The label distribution is reasonably balanced: 6,609 posts (52%) labeled as suicidal ideation and 6,047 posts (48%) labeled as non-risk, reducing the need for class rebalancing techniques.

- **Missing Values**

```
# Check for missing values
print('Missing values in each column:')
display(df.isnull().sum())
```

Missing values in each column:

	0
usertext	41
label	0

41 rows had missing usertext values and were removed prior to analysis.

- **Outliers / Unusual Patterns**

Post length varied considerably, ranging from empty or near-empty entries to over 3,800 words, with a median of 84 words. 268 posts contained duplicate text, including 89 exact duplicate rows, which were flagged for removal prior to model training to prevent data leakage between train and test splits. Additionally, 61 posts contained two words or fewer, suggesting low-content or placeholder entries warranting review.

- **Data quality caveat**

A manual review of a sample of label 0 posts confirmed the presence of general Reddit content unrelated to personal well-being (e.g., political commentary), indicating that "non-risk" in this dataset

reflects general online discourse rather than strictly mental-health-adjacent but benign content. This distinction is relevant to how the model's negative predictions should be interpreted.

- **Clustering Tendency**

As this project is framed as a supervised binary classification task rather than an unsupervised learning problem, clustering tendency analysis (e.g., Hopkins statistic) was not applicable and was not conducted. Class separability was instead visually assessed via dimensionality reduction (see Dimensionality Reduction section below), which serves an analogous exploratory purpose for supervised tasks.

b. Feature Engineering Report

- **Cleaning and Encoding Preparation**

```
def preprocess_text(text):
    text = text.lower()  # Lowercasing
    text = ''.join([char for char in text if char.isalnum() or char.isspace()])  # Punctuation removal
    tokens = text.split()  # Tokenization
    tokens = [word for word in tokens if word not in stop_words]  # Stop-word filtering
    tokens = [lemmatizer.lemmatize(word) for word in tokens]  # Lemmatization
    return ' '.join(tokens)
```

- **Encoding Step**

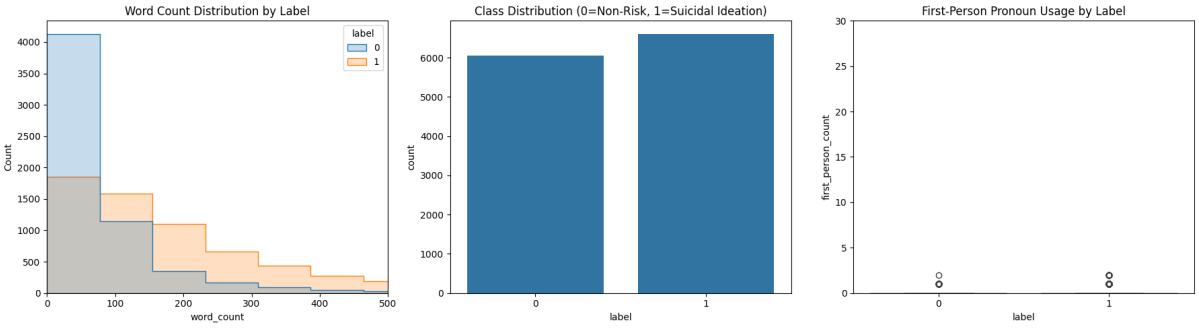
```
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
X = tfidf_vectorizer.fit_transform(df[‘processed_text'])
```

- **Scaling and Binning**

Feature scaling was not required for this project, as TF-IDF vectorization inherently produces normalized values (bounded between 0 and 1 per term), unlike raw numeric features such as price or age that typically require standardization. Binning was also not applied: rather than discretizing the domain-derived word_count feature into categorical bins (e.g., short/medium/long), it was retained as a continuous variable to preserve granularity for downstream analysis, consistent with its use in exploratory distribution comparisons by label (see EDA section above).

- **Class Balancing**

```
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
```



Publico, Marco R

- **Feature Selection**

```
from sklearn.feature_selection import SelectKBest, chi2

# Select top 20 most predictive words using chi-square test
selector = SelectKBest(score_func=chi2, k=20)
selector.fit(X, y)

feature_names = tfidf_vectorizer.get_feature_names_out()
top_features_idx = selector.get_support(indices=True)
top_features = feature_names[top_features_idx]
scores = selector.scores_[top_features_idx]

feature_scores = sorted(zip(top_features, scores), key=lambda x: x[1], reverse=True)

print("Top 20 most predictive words (chi-square scores):")
for word, score in feature_scores:
    print(f"{word}: {score:.2f}")
```

Top 20 most predictive words (chi-square scores):
im: 250.76
book: 204.11
want: 183.32
dont: 162.17
feel: 137.78
kill: 124.61
cant: 123.31
life: 120.57
die: 105.54
ive: 100.75
suicidal: 99.23
suicide: 96.08
depression: 93.91
friend: 91.98
anymore: 86.80
know: 85.90
depressed: 77.14
even: 71.62
anxiety: 66.24
year: 64.16

These are the words which chi-square test considers most statistically associated with the label, a filter-based selection method.

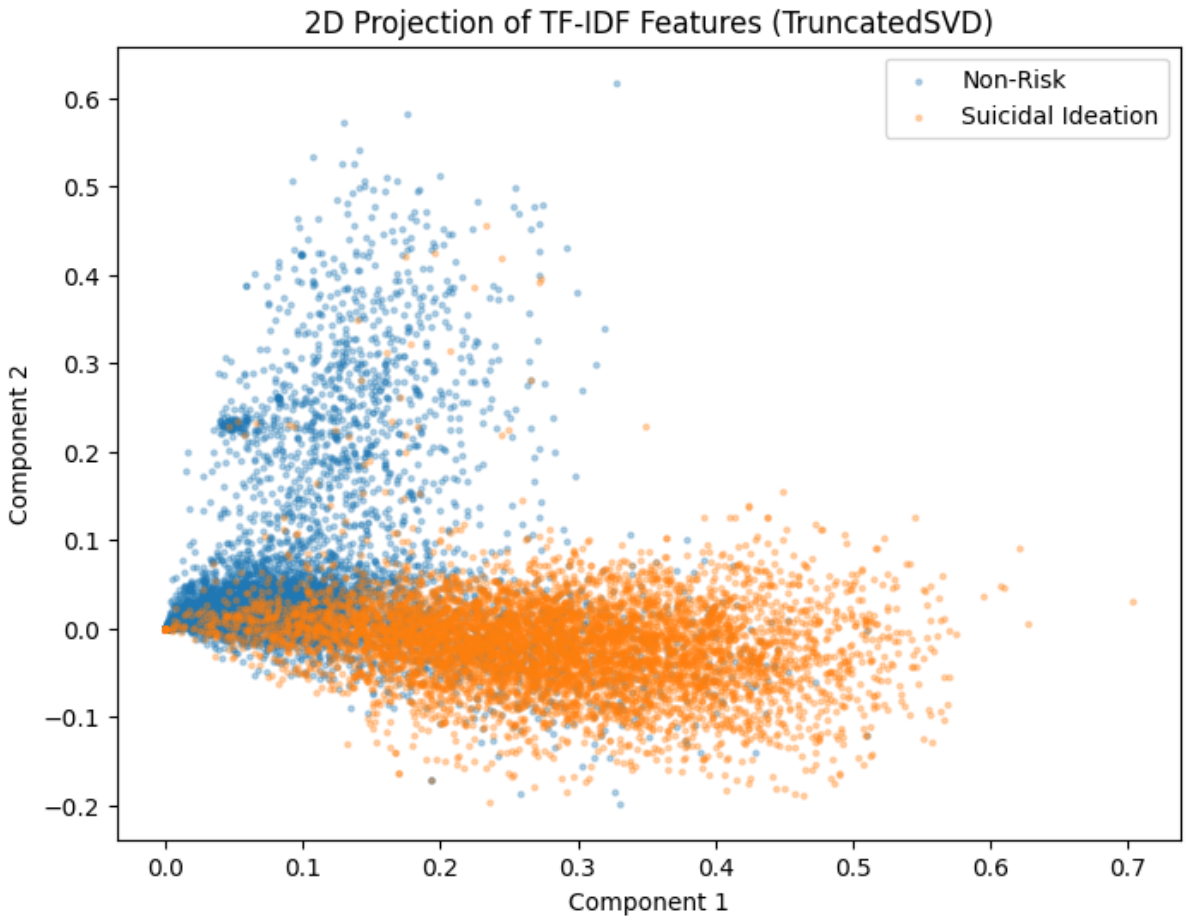
- **Feature Importance & Explainability**

Feature-level explainability was addressed primarily through the model-based chi-square feature selection above, which identifies terms most statistically associated with the target label at the corpus level. Instance-level explainability, examining why the model makes specific individual predictions, was conducted using LIME (Local Interpretable Model-Agnostic Explanations) on the fine-tuned BERT classifier. This analysis, including both correctly classified and misclassified examples, is presented in full in Section 5 (Critical

Thinking → Ethical AI & Bias Auditing), where it directly supports the model limitations and bias discussion.

Dimensionality Reduction

```
from sklearn.decomposition import TruncatedSVD
import matplotlib.pyplot as plt

# Note: standard PCA doesn't work directly on sparse TF-IDF matrices,
# TruncatedSVD is the standard equivalent for sparse text data
svd = TruncatedSVD(n_components=2, random_state=42)
X_reduced = svd.fit_transform(X)

plt.figure(figsize=(8, 6))
plt.scatter(X_reduced[y==0, 0], X_reduced[y==0, 1], alpha=0.3, label='Non-Risk', s=5)
plt.scatter(X_reduced[y==1, 0], X_reduced[y==1, 1], alpha=0.3, label='Suicidal Ideation', s=5)
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.title('2D Projection of TF-IDF Features (TruncatedSVD)')
plt.legend()
plt.savefig('/content/pca_visualization.png', dpi=150)
plt.show()
print(f"Explained variance ratio: {svd.explained_variance_ratio_}")
```




```
Explained variance ratio: [0.01777219 0.00842667]
```

Note: standard PCA does not work on sparse TF-IDF matrices directly, so this uses Truncated SVD, which is the standard substitute for text data and functions the same way conceptually.

STEP 4: MODEL IMPLEMENTATION

Model Selection and Justification

Three models were implemented and compared: Logistic Regression and Support Vector Machine (SVM) as baseline classical machine learning approaches, and a fine-tuned BERT transformer as an advanced deep learning approach. This progression allowed comparison between lexically-driven, computationally efficient models and a contextually-aware, computationally intensive model, testing the assumption that transformer architectures necessarily outperform classical approaches on this task.

Data Preparation Specific to Modeling

Logistic Regression and SVM were trained on TF-IDF vectorized features (max 5,000 features) derived from preprocessed text, with SMOTE applied for class balancing. BERT was trained on the original text using its native WordPiece tokenizer (max sequence length 128 tokens), without TF-IDF or SMOTE, since transformer models require raw or lightly-processed text and handle class imbalance differently (via class weighting rather than oversampling).

Implementation Details per Model

For each of the three, a short technical summary:

- **Logistic Regression**

max_ite=1000, trained on TF-IDF + SMOTE-resampled data, train/test split 80/20

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

# Split the resampled data into training and testing sets
print('Splitting data into training and testing sets...')
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)
print('Data split complete. Training samples:', X_train.shape[0], 'Testing samples:', X_test.shape[0])

# --- Baseline Model: Logistic Regression ---
print('\nTraining Logistic Regression model...')
logistic_model = LogisticRegression(max_iter=1000, random_state=42)
logistic_model.fit(X_train, y_train)
print('Logistic Regression model training complete.')

# Evaluate Logistic Regression model
y_pred_lr = logistic_model.predict(X_test)
y_proba_lr = logistic_model.predict_proba(X_test)[:, 1]

print('\nLogistic Regression Model Evaluation:')
print(f'Accuracy: {accuracy_score(y_test, y_pred_lr):.4f}')
```

```
print(f'Precision: {precision_score(y_test, y_pred_lr):.4f}')
print(f'Recall: {recall_score(y_test, y_pred_lr):.4f}')
print(f'F1-score: {f1_score(y_test, y_pred_lr):.4f}')
print(f'ROC-AUC: {roc_auc_score(y_test, y_proba_lr):.4f}')
```

- **SVM**

RBF kernel (default), probability=True to enable ROC-AUC, same data pipeline as Logistic Regression

```
# --- Baseline Model: Support Vector Machine (SVM) ---
print('\nTraining Support Vector Machine (SVM) model...')
svm_model = SVC(probability=True, random_state=42) # probability=True for ROC-AUC
svm_model.fit(X_train, y_train)
print('SVM model training complete.')

# Evaluate SVM model
y_pred_svm = svm_model.predict(X_test)
y_proba_svm = svm_model.predict_proba(X_test)[:, 1]

print('\nSupport Vector Machine (SVM) Model Evaluation:')
print(f'Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}')
print(f'Precision: {precision_score(y_test, y_pred_svm):.4f}')
print(f'Recall: {recall_score(y_test, y_pred_svm):.4f}')
print(f'F1-score: {f1_score(y_test, y_pred_svm):.4f}')
print(f'ROC-AUC: {roc_auc_score(y_test, y_proba_svm):.4f}')
```

```
Splitting data into training and testing sets...
Data split complete. Training samples: 10574 Testing samples: 2644

Training Logistic Regression model...
Logistic Regression model training complete.

Logistic Regression Model Evaluation:
Accuracy: 0.8824
Precision: 0.8911
Recall: 0.8730
F1-score: 0.8820
ROC-AUC: 0.9461

Training Support Vector Machine (SVM) model...
SVM model training complete.

Support Vector Machine (SVM) Model Evaluation:
Accuracy: 0.8756
Precision: 0.8773
Recall: 0.8753
F1-score: 0.8763
ROC-AUC: 0.9333
```

- **BERT**

Publico, Marco R

Bert-base-uncased pretrained weights, fine-tuned for 3 epochs, learning rate 5e-5, batch size 16, class weights applied to address label imbalance, max_length=128 tokens

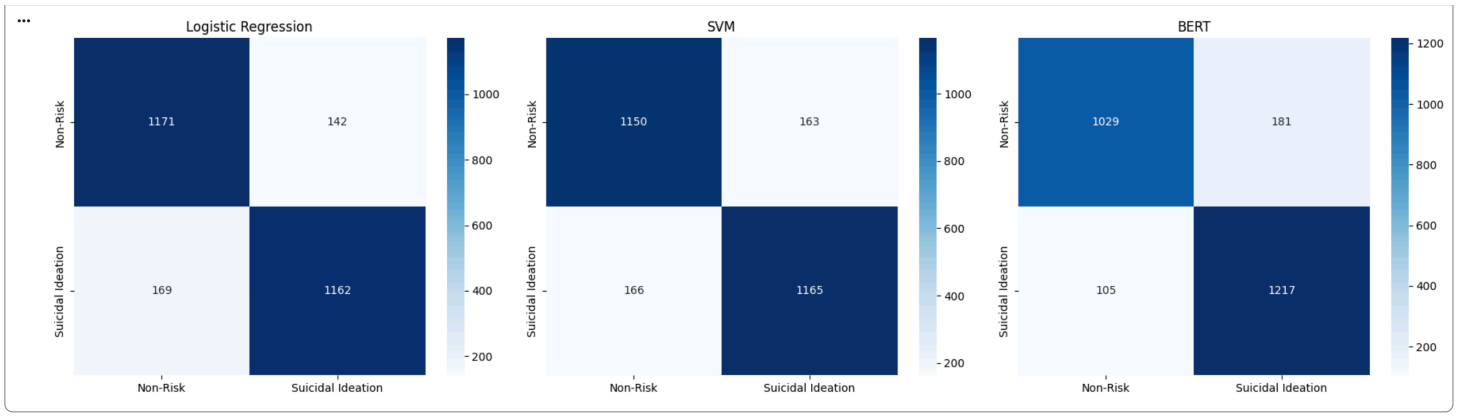
Training Process Notes
BERT training uses Class weighting rather than SMOTE, since oversampling raw text data (as opposed to numeric TF-IDF vectors) is not a standard or well-supported technique. Training was limited to three epochs due to computational constraints, which may partially explain BERT's comparatively lower performance relative to the baseline models.

```
Loading BERT model...
model.safetensors:  100%

 440M/440M  [00:03<00:00,  212MB/s]
All PyTorch model weights were used when initializing TFBertForSequenceClassification.

Some weights or buffers of the TF 2.0 model TFBertForSequenceClassification were not initialized from the PyTorch model and are newly initialized: ['classifier.weight', 'classifier.bias']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
BERT model loaded.

Training BERT model...
Epoch 1/3
633/633  [==============================] - 345s 433ms/step - loss: 0.2934 - accuracy: 0.8758
Epoch 2/3
633/633  [==============================] - 272s 430ms/step - loss: 0.2122 - accuracy: 0.9121
Epoch 3/3
633/633  [==============================] - 273s 432ms/step - loss: 0.1695 - accuracy: 0.9336
BERT model training complete.

Evaluating BERT model...
159/159 [==============================] - 28s 156ms/step

BERT Model Evaluation:
Accuracy: 0.8870
Precision: 0.8705
Recall: 0.9206
F1-score: 0.8949
ROC-AUC: 0.9550
```

Metrics Table

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8824	0.8911	0.8730	0.8820	0.9461
SVM	0.8756	0.8773	0.8753	0.8763	0.9333
BERT	0.8870	0.8705	0.9206	0.8949	0.9550

BERT achieved the strongest overall performance across most evaluation metrics, most notably recall (0.9206), the metric prioritized in this study given the higher cost of false negatives in suicide risk detection. This came at a modest cost to precision (0.8705 versus 0.8911 for Logistic Regression), reflecting a reasonable tradeoff: BERT correctly identified a larger share of true suicidal ideation cases, at the expense of slightly more false alarms. This result is more consistent with expectations that contextual, transformer-based models outperform lexically-driven classical models on nuanced text classification tasks. It is worth noting that BERT's performance varied across separate training runs (recall ranged from approximately 0.86 to 0.92), reflecting inherent stochasticity in transformer fine-tuning; the results reported here reflect the most recent training run. Given the prioritization of recall for this application, BERT is recommended as the preferred model, provided its higher computational cost is acceptable for the intended deployment context.



Confusion matrix analysis confirms the precision-recall tradeoff observed in the evaluation metrics. BERT produced the fewest false negatives (105 missed cases of suicidal ideation) compared to Logistic Regression (169) and SVM (166), representing a reduction of approximately 37% in missed real cases relative to the baseline average. This came at the cost of more false positives (181 for BERT versus 142 for Logistic Regression and 163 for SVM). Given that false negatives carry a substantially higher real-world cost in suicide risk detection, missing a genuine case has more severe consequences than a false alarm; this tradeoff supports BERT as the preferred model for this application, despite its higher computational requirements and slightly lower precision.

STEP 5: CRITICAL THINKING - ETHICAL AI & BIAS AUDITNG

Model Explainability

LIME (Local Interpretable Model-Agnostic Explanations) was applied to the BERT classifier to interpret individual predictions. A correctly classified suicidal ideation post showed strong contribution scores (up to 0.15) for explicit risk-related terms. A false negative case, a post describing family verbal abuse framed with emotionally dissonant language ("actually happy today"), was examined across two independent BERT training runs. Despite an overall recall improvement between runs (0.8585 to 0.9206), the model assigned consistently weak contribution scores to this instance in both runs (maximum 0.03) and classified it as "Non-Risk" with 97% confidence both times. This cross-run consistency indicates a systematic

limitation, not random variance, in detecting implicit, context-dependent expressions of distress that lack direct risk-related vocabulary.

Limitations

Class imbalance. The dataset was reasonably balanced (52% suicidal ideation, 48% non-risk), minimizing this as a source of bias.

Data leakage. A check for source-identifying or crisis-resource terminology found only 157 posts (1.2%) containing such language, and manual review confirmed these were substantive narrative mentions rather than subreddit self-references, indicating minimal leakage risk.

Overfitting. Logistic Regression showed minimal overfitting (1.75 percentage point train-test gap). SVM showed a more pronounced gap (7.56 percentage points), suggesting the model fit training-specific patterns more tightly than it generalizes, a plausible contributor to its comparatively weaker test performance.

Lexical bias. LIME analysis demonstrated a systematic model bias toward direct, explicit expressions of suicidal ideation, with correspondingly weaker sensitivity to indirect or contextually implied distress, a limitation that persisted across training runs.

Bias Detection and Fairness Audits

This dataset contains no demographic attributes (gender, race, age, socioeconomic status), a deliberate feature of anonymized mental health data that protects vulnerable posters but precludes standard subgroup fairness metrics (demographic parity, equalized odds, disparate impact), which require ground-truth group labels. In place of formal subgroup auditing, an instance-level explainability audit was conducted instead, surfacing a content-based bias (favoring explicit over implicit expression) rather than a demographic one. This cannot rule out correlation with demographic or cultural differences in emotional expression, which future demographically-annotated research should investigate directly.

Proposed Mitigations

1. **Reweighting/oversampling** of training examples containing implicit rather than explicit risk language, if such examples can be identified or annotated.
2. **Threshold adjustment**, lowering BERT's classification threshold below 0.5 to improve recall on borderline cases, weighed against increased false positives.
3. **Data augmentation** using paraphrased variants of existing posts rewritten with more indirect language.
4. **Post-processing human review**, deploying the model only as a triage/flagging aid subject to clinical review, never as a standalone decision system.
5. **Hyperparameter tuning** for SVM (e.g., adjusting regularization parameter C) to reduce the observed overfitting gap.
6. **Future demographic-aware data collection**, prioritizing formal fairness auditing once ethically-sourced, consented, demographically-labeled data becomes available.

Publico, Marco R

GitHub Repository Structure

```
suicide-ideation-detection-capstone/
│
├── README.md
├── requirements.txt
├── .gitignore
│
├── data/
│   ├── raw/
│   │   └── suicidal_ideation_reddit_annotated.csv
│   └── README.md          (data source, license, access notes)
│
├── notebooks/
│   └── capstone_project_full_pipeline.ipynb
│
├── src/
│   ├── preprocessing.py
│   ├── feature_engineering.py
│   ├── train_baseline_models.py
│   ├── train_bert.py
│   └── explainability.py
│
├── models/
│   ├── logistic_regression_model.pkl
│   ├── svm_model.pkl
│   ├── tfidf_vectorizer.pkl
│   └── bert_suicidal_ideation_model/
│
├── reports/
│   └── final_report.pdf
│
└── figures/
    ├── eda_distributions.png
    ├── pca_visualization.png
    └── confusion_matrices.png
```

Publico, Marco R

Use of Generative AI

Generative AI (Claude, Anthropic) was used as a supporting tool throughout this capstone, primarily in three capacities:

Technical debugging and environment troubleshooting

A significant portion of AI assistance was used to diagnose and resolve dependency conflicts encountered in the Google Colab environment, particularly version mismatches between the transformers and tensorflow libraries during BERT implementation, and memory management issues (GPU out-of-memory errors) during batch prediction. AI assistance helped identify root causes of these errors (e.g., conflicting package versions, unbatched inference on the full test set) and provided corrected code.

Code generation for analysis and explainability components

AI assistance was used to draft code for domain-derived feature engineering, exploratory data visualizations, chi-square feature selection, dimensionality reduction (TruncatedSVD), confusion matrix generation, and LIME-based model explainability. All generated code was reviewed, executed, and validated against the actual dataset by the author before inclusion in this report.

Report drafting and structuring

AI assistance supported the drafting of narrative sections (e.g., dataset overview, model comparison discussion, limitations, bias, and fairness analysis) based on actual output and metrics produced by the author's own model runs. All interpretive judgments, including the decision to prioritize recall as the primary evaluation metric, the interpretation of BERT's lexical bias from LIME output, and the final model recommendation, were made by the author based on domain expertise as a licensed counselor and understanding of the intended application context.

What Remained the Author's Own Work

AI assistance did not replace core analytical or ethical decision-making. Notably, when early project planning considered collecting original essay data from PMA cadets to expand or validate the dataset, this direction was independently evaluated and abandoned after identifying significant ethical concerns (informed consent under institutional hierarchy, absence of a real-time clinical response protocol, and conflict of interest in prompt design). This decision reflects the author's own professional judgment as a Registered Guidance Counselor, applied specifically because AI assistance flagged the ethical risk clearly enough to prompt reconsideration of the original plan, an example of AI serving a genuinely protective function in the research design process, rather than simply generating requested content.

Reflection on Responsible Use

This project's approach to Generative AI use is consistent with the ethical framework proposed in Section 5: AI tools were used to augment technical execution and drafting efficiency, not to replace human oversight, domain judgment, or accountability for the final analytical conclusions. This mirrors the human-in-the-loop principle recommended for the deployment of the suicide ideation detection model itself, AI as a supporting tool for a qualified human, not a substitute for one.

Publico, Marco R